

Programación de Bases de Datos (RA5)

Procedimientos, Funciones y Triggers en Sakila

Gestión de Bases de Datos

17 de abril de 2026

FP Madrid - Especialidad Informática

1. ¿Por qué programar en la BBDD?
2. Procedimientos Almacenados
3. Funciones de Usuario
4. Estructuras de Control
5. Disparadores (Triggers)

¿Por qué programar en la BBDD?

¿Por qué programar en la BBDD?

- **Automatización:** Tareas repetitivas que el SGBD hace solo.
- **Seguridad (Blindaje):** Las reglas de negocio se quedan en la BBDD.
- **Eficiencia:** Menos tráfico de red (llamadas ligeras).
- **Centralización:** Un solo cambio afecta a todas las Apps.

El concepto clave: DELIMITER

Cambiamos el fin de orden (;) por otro (//) para que MySQL no ejecute el código antes de terminar de escribir el bloque completo.

Procedimientos Almacenados

Stored Procedures: Ejemplos i

1. Sin parámetros:

Listing 1: src/proc_saludo.sql

```
DELIMITER //  
CREATE PROCEDURE saludo()  
BEGIN  
    SELECT '¡Hola clase de DAM!' AS mensaje;  
END //  
DELIMITER ;  
  
CALL saludo();
```

2. Parámetro de entrada (IN):

Listing 2: src/proc_buscar_actor.sql

```
DELIMITER //
CREATE PROCEDURE buscar_actor(IN p_apellido VARCHAR(45))
BEGIN
    SELECT first_name, last_name FROM actor
    WHERE last_name LIKE CONCAT(p_apellido, '%');
END //
DELIMITER ;

CALL buscar_actor('Jackman');
```

3. Parámetro de salida (OUT):

Listing 3: src/proc_contar_peliculas.sql

```
DELIMITER //  
CREATE PROCEDURE contar_peliculas(OUT p_total INT)  
BEGIN  
    SELECT COUNT(*) INTO p_total FROM film;  
END //  
DELIMITER ;  
  
CALL contar_peliculas(@resultado);  
SELECT @resultado;
```


4. Ejemplo Sakila (IN y OUT):

Listing 4: src/proc_get_actor_stats.sql

```
DELIMITER //
CREATE PROCEDURE get_actor_stats(IN p_actor_id INT, OUT p_films INT, OUT
    p_avg_len DECIMAL(10,2))
BEGIN
    SELECT COUNT(*), AVG(length) INTO p_films, p_avg_len
    FROM film JOIN film_actor USING(film_id) WHERE actor_id = p_actor_id;
END //
DELIMITER ;
```

Funciones de Usuario

Características de la Función

Característica	Significado
DETERMINISTIC	Mismo input → Mismo output
NOT DETERMINISTIC	Usa NOW(), RAND()
READS SQL DATA	Solo hace SELECT
MODIFIES SQL DATA	Hace INSERT/UPDATE/DELETE

Importante: Error 1418

MySQL prohíbe crear funciones si no se declaran estas etiquetas por seguridad en el log binario.

1. Operación Matemática:

Listing 5: src/func_calcular_iva.sql

```
DELIMITER //  
CREATE FUNCTION calcular_iva(p_precio DECIMAL(10,2))  
RETURNS DECIMAL(10,2) DETERMINISTIC  
BEGIN  
    RETURN p_precio * 1.21;  
END //  
DELIMITER ;
```

2. Formateo de Texto:

Listing 6: src/func_nombre_completo.sql

```
DELIMITER //  
CREATE FUNCTION nombre_completo(p_nombre VARCHAR(45), p_apellido VARCHAR(45))  
RETURNS VARCHAR(100) DETERMINISTIC  
BEGIN  
    RETURN CONCAT(p_apellido, ' ', p_nombre);  
END //  
DELIMITER ;
```

3. Consulta de BBDD:

Listing 7: src/func_dias_alquiler.sql

```
DELIMITER //
CREATE FUNCTION dias_alquiler(p_rental_id INT)
RETURNS INT READS SQL DATA
BEGIN
    DECLARE v_dias INT;
    SELECT DATEDIFF(return_date, rental_date) INTO v_dias
    FROM rental WHERE rental_id = p_rental_id;
    RETURN v_dias;
END //
DELIMITER ;
```

4. Ejemplo Sakila Complejo:

Listing 8: src/func_inventory_in_stock.sql

```
DELIMITER //
CREATE FUNCTION inventory_in_stock(p_inventory_id INT)
RETURNS BOOLEAN READS SQL DATA
BEGIN
    DECLARE v_rentals INT;
    DECLARE v_out     INT;
    SELECT COUNT(*) INTO v_rentals FROM rental WHERE inventory_id =
        p_inventory_id;
    IF v_rentals = 0 THEN RETURN TRUE; END IF;
    SELECT COUNT(rental_id) INTO v_out
    FROM inventory LEFT JOIN rental USING(inventory_id)
    WHERE inventory.inventory_id = p_inventory_id AND rental.return_date IS NULL
    ;
    IF v_out > 0 THEN RETURN FALSE; ELSE RETURN TRUE; END IF;
END //
DELIMITER ;
```

Estructuras de Control

1. Condicional IF:

Listing 9: src/ctrl_if.sql

```
IF p_edad >= 18 THEN
    SET p_mensaje = 'Mayor de edad';
ELSE
    SET p_mensaje = 'Menor de edad';
END IF;
```

2. Estructura CASE:

Listing 10: src/ctrl_case.sql

```
CASE p_categoria_id
    WHEN 1 THEN SET p_nombre_cat = 'Acción';
    WHEN 2 THEN SET p_nombre_cat = 'Animación';
    ELSE SET p_nombre_cat = 'Otros';
END CASE;
```

3. Bucle WHILE:

Listing 11: src/ctrl_while.sql

```
DECLARE i INT DEFAULT 1;
WHILE i <= 5 DO
    INSERT INTO tabla_log(msg) VALUES (CONCAT('Paso ', i));
    SET i = i + 1;
END WHILE;
```

4. Bucle LOOP / LEAVE:

Listing 12: src/ctrl_loop.sql

```
mi_bucle: LOOP
    IF v_error = TRUE THEN
        LEAVE mi_bucle;
    END IF;
    -- Realizar acciones...
END LOOP mi_bucle;
```

5. Caso Real: Bucle Masivo (Logística):

Listing 13: src/ctrl_masivo.sql

```
DELIMITER //
CREATE PROCEDURE GenerarEnviosMasivos()
BEGIN
    DECLARE i INT DEFAULT 0;
    WHILE i < 100000 DO
        INSERT INTO envios (tracking_number, cliente_id, f_salida, importe_envio
        )
        VALUES (CONCAT('TRK-', FLOOR(RAND()*999999999)),
                FLOOR(1 + RAND() * 500),
                DATE_FORMAT(DATE_ADD('2025-01-01', INTERVAL i MINUTE),
                            ELT(1 + FLOOR(RAND() * 4), '%d/%m/%Y', '%Y-%m-%d',
                                '%d-%m-%y', '%y/%m/%d')),
                CONCAT(ROUND(RAND()*500, 2), ' EUR'));
        SET i = i + 1;
        IF i % 20000 = 0 THEN
            SELECT CONCAT('... ', i, ' envios procesados.') AS Progreso;
        END IF;
    END WHILE;
END //
```

```
DELIMITER ;
```

Disparadores (Triggers)

Disparadores: Conceptos Clave

- **BEFORE:** Validar/Cambiar datos antes de guardar.
- **AFTER:** Auditoría o sincronización posterior.

Operación	INSERT	UPDATE	DELETE
NEW	✓	✓	—
OLD	—	✓	✓

1. Validar (BEFORE INSERT):

Listing 14: src/trig_capitalizar.sql

```
DELIMITER //  
CREATE TRIGGER capitalizar_apellido BEFORE INSERT ON actor  
FOR EACH ROW  
BEGIN  
    SET NEW.last_name = UPPER(NEW.last_name);  
END //  
DELIMITER ;
```

2. Auditoría (AFTER INSERT):

Listing 15: src/trig_log_cliente.sql

```
DELIMITER //  
CREATE TRIGGER log_nuevo_cliente AFTER INSERT ON customer  
FOR EACH ROW  
BEGIN  
    INSERT INTO logs(accion, id_cliente) VALUES ('NUEVO', NEW.customer_id);  
END //  
DELIMITER ;
```


3. Impedir acción (SIGNAL):

Listing 16: src/trig_signal.sql

```
DELIMITER //  
CREATE TRIGGER no_borrar_actores BEFORE DELETE ON actor  
FOR EACH ROW  
BEGIN  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Prohibido borrar actores';  
END //  
DELIMITER ;
```

4. Sincronización (AFTER INSERT):

Listing 17: src/trig_ins_film.sql

```
DELIMITER //
CREATE TRIGGER ins_film AFTER INSERT ON film
FOR EACH ROW
BEGIN
    INSERT INTO film_text (film_id, title, description)
    VALUES (NEW.film_id, NEW.title, NEW.description);
END //
DELIMITER ;
```

¿Dudas?

Próxima sesión: Cursores y Handlers